Affordable and Effective Analyses of Optical Properties of Fluids using an Arduino

PHY 3902 Laboratory Experiment 5

Author(s): Michelle Johnstone, Patrick Geraghty
Affiliation: University of Ottawa
Date: November 30, 2025

1 Abstract

Diffraction is a powerful method of analyzing the behaviour of waves including how they propagate and interact with materials. For light, common ways of observing this behaviour are single slit, double slit, and diffraction grating experiments which can be used to analyze the spectrum of an incident beam. When a beam enters a medium, the velocity and wavelength of the light change; the ratio of the speed of light in a vacuum to this velocity in the medium is called the index of refraction and is unique to a medium's optical density. This experiment focuses on using two lasers of known wavelengths to analyze the refractive indices of a variety of media with theoretical indices ranging from about 1 to 1.5 at standard atmosphere, temperature, and pressure. These materials were placed between a diffraction grating and a screen to analyze the change in the light's velocity in the medium by observing the change in the diffraction pattern. This change in the degrees of diffraction was recorded using a motorized light-dependent resistor controlled by an Arduino, from which the refractive indices of air, water, ethanol, and olive oil were calculated to within 3.2%, 6.8%, 10.4%, and 20.7% of their expected values respectively. These results demonstrate that diffraction-based refractometry provides an effective and accessible method of determining select optical properties of non-opaque media.

2 Introduction

When monochromatic light passes through a diffraction grating, it splits into a characteristic pattern of bright and dark fringes due to wave interference. Diffraction, first described by Francesco Grimaldi in 1660, is the phenomenon where waves bend and spread around obstacles or through apertures. The extent of this bending depends on the ratio of the wavelength to the size of the opening.

A diffraction grating consists of many parallel slits with spacing comparable to the wavelength of light. When light passes through these slits, the emerging waves interfere with each other, creating a pattern of constructive and destructive interference. Constructive interference produces bright fringes (maxima), while destructive interference creates dark regions (minima). The central, undeviated beam is the zeroth-order maximum, and subsequent symmetrically spaced fringes are the higher-order maxima.

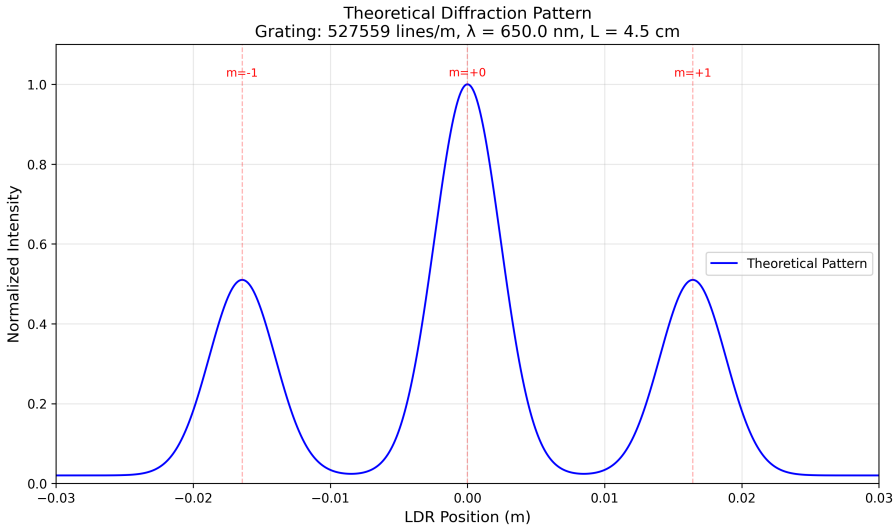


Figure 1: Theoretical diffraction pattern intensity profile measured through air using the 650 nm laser. The central peak (zeroth order) shows maximum intensity with symmetric first-order fringes on either side, demonstrating the characteristic diffraction pattern produced by the grating.

Constructive interference occurs when the path difference between waves from adjacent slits is an integer multiple m of the wavelength λ. For a diffraction grating with slit spacing d, this path difference forms the opposite side of a right triangle where d is the hypotenuse. The angle θ of the mth-order fringe relative to the normal is given by:

$$\theta_{air} = \arcsin\left(\frac{m\lambda}{d}\right) \tag{1}$$

The refractive index of a medium is defined as the ratio of the speed of light in a vacuum c to the speed of light in that medium v. For linear media, this index depends on optical density and can be affected by factors such as temperature and pressure. Since the frequency f of light remains constant as it propagates through different media (being determined by the source), we can relate the wavelength in vacuum λ_0 to the wavelength in a medium λ.

Starting with the fundamental relationship $c = f\lambda_0$ for light in vacuum and $v = f\lambda$ for light in a medium, we can solve for frequency and equate them:

$$\frac{c}{\lambda_0} = \frac{v}{\lambda} \tag{2}$$

This yields the relationship between wavelength and refractive index:

$$\lambda = \frac{\lambda_0}{n} \quad \text{where} \quad n = \frac{c}{v} \tag{3}$$

Combining this with equation (1) allows us to determine the refractive index from the diffraction angle:

$$n = \frac{\lambda_0 m}{d\sin(\theta)} \tag{4}$$

This formula is valid when light travels through a single uniform medium. However, in this experiment, the diffraction grating remains in air while the diffracted light passes

through a liquid-filled cuvette before reaching the detector. This multi-medium optical path requires a more detailed analysis.

Since the grating cannot be submerged, it remains in air ($n_{air} \approx 1$), and equation (1) determines the initial diffraction angle in air. The light then enters the liquid medium, where it refracts according to Snell's law:

$$n_1 \sin(\theta_1) = n_2 \sin(\theta_2) \tag{5}$$

As the beam passes through the liquid, refraction bends it toward the normal if the liquid is more optically dense than air, or away from the normal if less dense. This refraction causes a lateral displacement (Δx) of the fringe position at the detector. For accurate measurements, any air gap greater than 1 mm between the grating and the liquid must be accounted for, as such gaps introduce significant systematic errors.

The lateral displacement of a fringe depends on the path length through each medium and the refractive indices involved. For a fringe at position x_i, the geometry of the beam path gives:

$$x_i = p \tan(\theta_i) \tag{6}$$

where p is the path length and θ_i is the propagation angle in that medium relative to its position when propagating only through air.

For the two-medium system (air and liquid) with the beam initially normal to the grating, the displacement of the ith-order fringe can be expressed as:

$$\Delta x = p \sin(\theta_{\text{air}}) \left(\frac{1}{\sqrt{1 - \sin^2(\theta_{\text{air}})}} - \frac{1}{\sqrt{n_{liq}^2 - \sin^2(\theta_{\text{air}})}} \right) \tag{7}$$

where p is the liquid path length. This equation assumes normal incidence on the grating and parallel cuvette walls, and is not suitable for more than two media or non-normal incidence without modification.

Solving equation (7) for the refractive index yields:

$$n_{liq} = \sin(\theta_{\text{air}}) \sqrt{1 + \frac{p^2 (1 - \sin^2(\theta_{\text{air}}))}{(p \sin(\theta_{\text{air}}) - \Delta x \sqrt{1 - \sin^2(\theta_{\text{air}})})^2}} \tag{8}$$

By measuring the fringe positions with an empty cuvette (baseline) and comparing them to the positions with the filled cuvette, the effective displacement Δx can be determined. This approach eliminates the need to account for the optical effects of the cuvette walls, as they are present in both measurements and cancel out when calculating the displacement.

Once the refractive index is determined, the wavelength of light in the liquid can be calculated by combining equations (2) and (8):

$$\lambda_{liq} = \frac{d}{m \sqrt{1 + \frac{p^2 (1 - \sin^2(\theta_{\text{air}}))}{(p \sin(\theta_{\text{air}}) - \Delta x \sqrt{1 - \sin^2(\theta_{\text{air}})})^2}}} \tag{9}$$

This wavelength shift is most noticeable in materials with high refractive indices. The goal of this experiment is to validate this cost-effective diffraction-based method for measuring the refractive indices of common liquids.

3 Methods

The experimental apparatus consisted of a laser source, a diffraction grating, a sample cuvette, and a light-dependent resistor (LDR) mounted on a stepper motor-driven linear track. The laser was positioned at one end of the optical table, with the LDR assembly at the opposite end. A diffraction grating was placed between them on an adjustable stand. The track length was designed to capture the first-order diffraction peaks, calculated based on the theoretical diffraction angle for air using equation (1). All light-sensitive components were enclosed in a black box to minimize ambient light interference and maximize the contrast between bright and dark fringes.

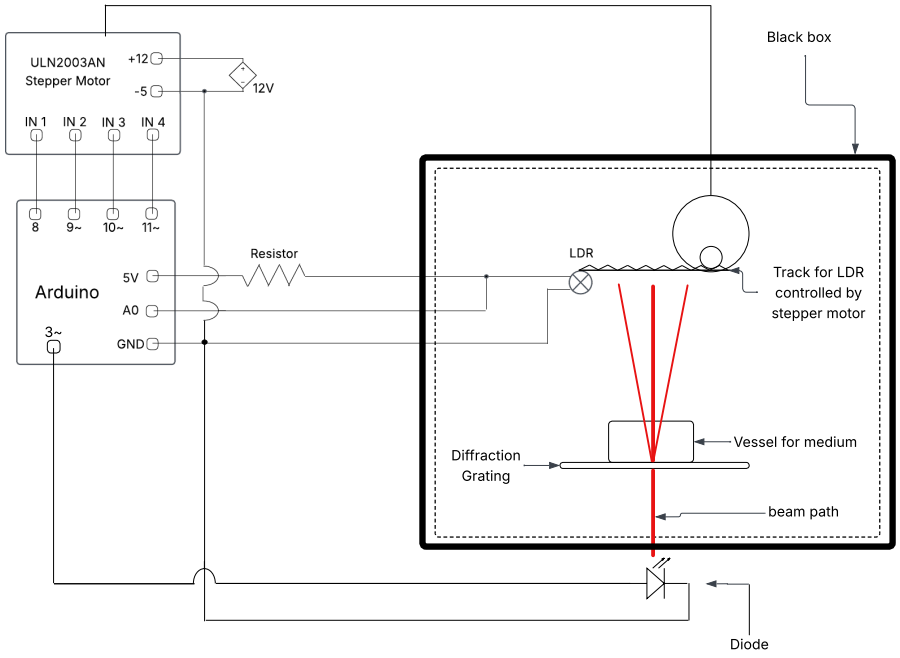


Figure 2: Diagram of the experimental setup showing the laser source, diffraction grating, cuvette, and LDR mounted on the linear track. The entire assembly is enclosed in a light-tight box to reduce ambient light interference.

The cuvette placement proved critical to measurement accuracy. Initial trials positioned the cuvette 1–2 mm from the diffraction grating, which produced refractive indices with 10–20% errors. Analysis revealed that air gaps greater than 1 mm introduce significant lateral shifts that must be accounted for in the theoretical model. To simplify the optical path and improve accuracy, the cuvette was repositioned flush against the grating, eliminating the air gap and allowing equation (3) to be directly applicable.

The experimental procedure began with control measurements to validate the setup. First, the diffraction pattern was measured through air alone to verify that the apparatus could correctly determine the refractive index of air ($n \approx 1.000$). Next, an empty cuvette was placed flush with the grating to establish a baseline measurement. This baseline accounts for the optical effects of the quartz cuvette walls, allowing the displacement due to the liquid alone to be isolated by comparison.

The choice of cuvette significantly impacted data quality. Initial measurements used a plastic semi-micro cuvette, but the protruding edges clipped the first-order diffraction

beams, distorting the fringe positions. Switching to a quartz macro cuvette with a wider aperture resolved this issue and substantially improved alignment and data collection.

Each test liquid (water, ethanol, and extra virgin olive oil) was measured in sequence, with the cuvette rinsed thoroughly with ethanol between samples to prevent cross-contamination. Liquids were tested in order of increasing refractive index, with consideration given to viscosity for efficient rinsing.

Two laser sources were employed to assess wavelength-dependent effects: a 650 nm red laser and a 532 nm green laser. For each measurement, the LDR scanned across the diffraction pattern in 0.2 mm increments. At each position, 20 resistance readings were taken and averaged to reduce noise and improve precision. The inverse resistance (proportional to light intensity) was plotted against the LDR position on the track.

The LDR detection system produced broad, asymmetric intensity peaks rather than sharp maxima. To determine the center of each peak, the positions where the intensity exceeded a threshold value were identified, and the midpoint of this span was calculated. This method provided consistent peak localization despite the inherent limitations of the LDR sensor.

The positions of the first-order peaks (both positive and negative diffraction orders) were determined from the intensity plots and compared to the baseline positions from the empty cuvette measurement. The displacement values (Δx) were then substituted into equation (8) to calculate the refractive index of each liquid. The measurements were repeated with both laser wavelengths to verify precision and identify any systematic wavelength-dependent errors.

4 Results and Discussion

4.1 Experimental Challenges and Equipment Optimization

Initial results for the calculation were found using a semi-micro cuvette. After many trials and measurements, all produced results below the accepted minimum refractive index in vacuum of 1.000. Many adjustments were made to the alignment of the setup and the measurements and calculations were redone and triple-checked, without much improvement. Finally, the plastic semi-micro cuvette was swapped for a quartz macro cuvette. Instant improvements were observed in the results, and it was determined that the protruding edges of the semi-micro cuvette were clipping the first order fringe beams exiting the liquid. This resulted in errors in all the calculations as the beams were diverted from their natural diffraction angles.

4.2 Refractive Index Measurements

Table 1 summarizes the refractive index measurements obtained for various liquids using both 650 nm (red) and 532 nm (green) laser sources. The measurements were conducted with the optimized quartz macro cuvette setup.

Table 1: Measured refractive indices for various liquids at SATP. Multiple trials were conducted for water, ethanol, and EVOO to assess reproducibility. Values are reported as mean ± standard deviation calculated from the two first-order fringes.

Liquid	**Trial**	**Wavelength (nm)**	**Measured n**	**Theoretical n**
Air	–	650	1.032 ± 0.236	1.000
Water	1	650	1.436 ± 0.335	1.333
Water	2	650	1.321 ± 0.280	1.333
Water	3	532	1.424 ± 0.211	1.333
Ethanol	1	650	1.750 ± 0.504	1.361
Ethanol	2	650	1.533 ± 0.197	1.361
Ethanol	3	532	1.220 ± 0.077	1.361
EVOO	1	650	1.185 ± 0.272	1.465–1.480
EVOO	2	650	1.161 ± 0.054	1.465–1.480
EVOO	3	532	1.175 ± 0.140	1.465–1.480

The raw intensity data collected by the LDR scanning system is shown in Figure 3 for water measured with the 532 nm laser. The plot clearly shows the central maximum (zeroth order) and the symmetric first-order diffraction peaks on either side, demonstrating the quality of the diffraction pattern despite the broad peak widths characteristic of LDR-based detection.

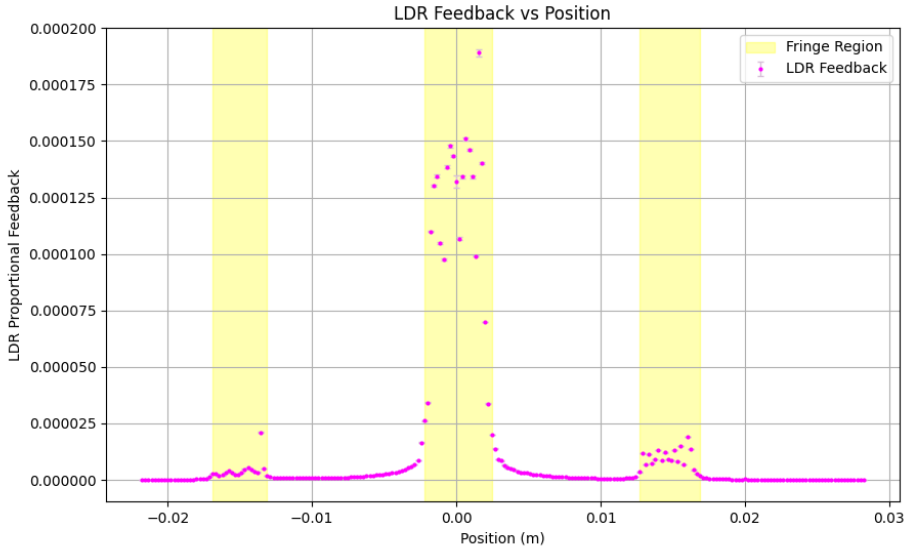


Figure 3: Representative intensity profile for water measured with the 532 nm laser. The inverse resistance (proportional to light intensity) is plotted as a function of LDR position along the track. The central peak corresponds to the zeroth-order fringe, with symmetric first-order peaks visible on either side.

Figure 4 presents representative intensity profiles for each liquid measured with the 650 nm laser, illustrating how the fringe positions shift with different refractive indices. The variation in peak separation between liquids demonstrates the sensitivity of the diffraction-based measurement technique.

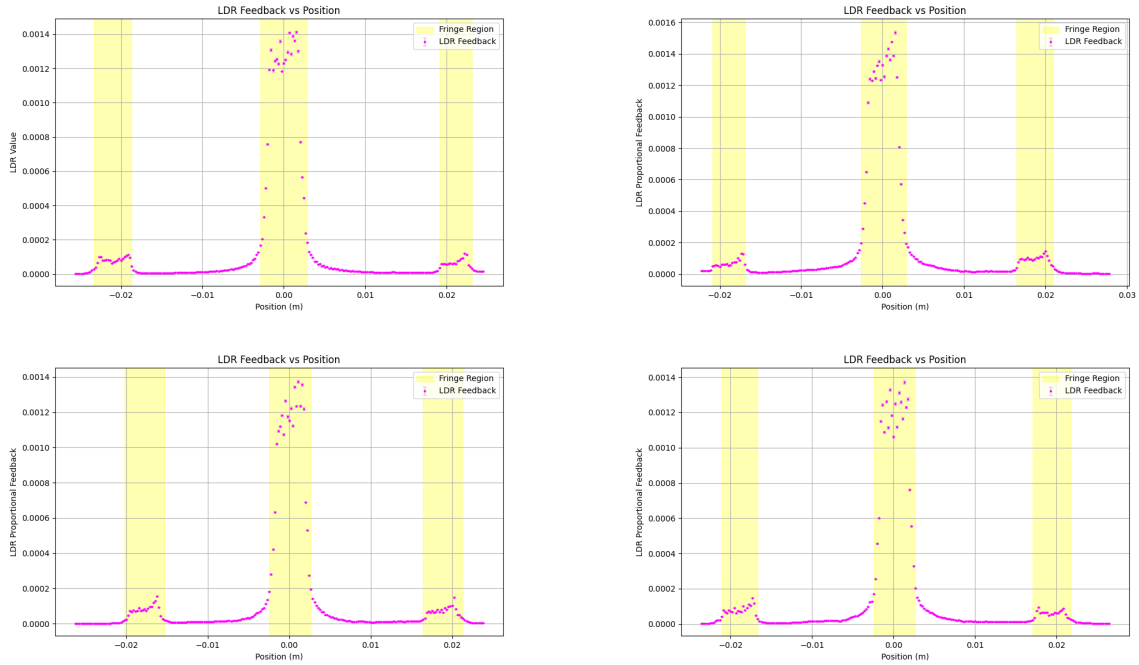


Figure 4: Comparison of intensity profiles for (top left) air baseline, (top right) water, (bottom left) ethanol, and (bottom right) extra virgin olive oil, all measured with the 650 nm laser using the optimized quartz macro cuvette setup. Note the variation in fringe positions and peak widths between different media.

4.3 Analysis of Results

4.3.1 Air Baseline Measurement

The refractive index of air at SATP was measured to be 1.032 ± 0.236 using the 650 nm laser. While this value is close to the theoretical value of 1.000, it carries a relatively large uncertainty of 23%. The measured fringe positions were symmetric about the center (21.11 mm for both orders), suggesting good alignment of the optical setup. The deviation from unity and the substantial uncertainty likely arise from experimental factors including fringe detection threshold sensitivity and the inherent limitations of the LDR-based measurement system.

4.3.2 Water Measurements

Water was measured multiple times throughout the experiment to assess reproducibility. Using the 650 nm laser, the first measurement yielded $n = 1.436 \pm 0.335$ while the second measurement with the improved setup gave $n = 1.321 \pm 0.280$. The 532 nm laser measurement produced $n = 1.424 \pm 0.211$. These values bracket the theoretical value of 1.333, with percent errors ranging from 1.0% to 7.7%. The improved precision with the green laser (14.8% relative uncertainty compared to 21.2–23.3% for red) suggests that shorter wavelengths may provide more distinct fringe patterns for this experimental configuration.

4.3.3 Ethanol Measurements

Ethanol presented more challenging results. The initial measurement with the old cuvette setup yielded $n = 1.750 \pm 0.504$, significantly higher than the theoretical value of 1.361 (28.6% error). After optimizing the cuvette placement, the 650 nm measurement improved to $n = 1.533 \pm 0.197$ (12.6% error). Interestingly, the 532 nm measurement gave $n = 1.220 \pm 0.077$, which underestimates the theoretical value by 10.4% but with notably better precision (6.3% relative uncertainty). The systematic underestimation with the green laser may indicate wavelength-dependent systematic errors in the fringe detection algorithm or optical path considerations.

4.3.4 Extra Virgin Olive Oil Measurements

Extra virgin olive oil (EVOO), with the highest theoretical refractive index in this study (1.465–1.480), consistently produced measurements lower than expected. The three measurements yielded $n = 1.185 \pm 0.272$ (650 nm, v1), $n = 1.161 \pm 0.054$ (650 nm, v2), and $n = 1.175 \pm 0.140$ (532 nm, v2), corresponding to approximately 19.0–20.7% underestimation. The observed fringe distances were 14.22–14.44 mm for the old setup and 18.89–19.33 mm for the new setup with 650 nm light, significantly different from water (19.78–20.22 mm) despite olive oil having a higher refractive index. This counterintuitive result suggests potential systematic errors in equation (9) when applied to materials with substantially higher refractive indices, possibly related to the simplifying assumptions made in the theoretical derivation or to absorption and scattering effects in the olive oil.

4.4 Discussion of Systematic Errors

Several systematic factors contribute to the observed discrepancies:

Fringe Detection: The LDR-based detection system measures broad, asymmetric intensity peaks rather than sharp maxima. The center of each fringe was estimated from the span of positions exceeding a threshold intensity, introducing subjectivity and potential bias in peak localization. The fringes could likely be measured more accurately with a stationary detection system, such as a photodiode array or high-resolution camera.

Wavelength Effects: The green laser (532 nm) generally produced lower uncertainties than the red laser (650 nm), likely due to better spatial resolution of the diffraction pattern. However, systematic offsets between wavelengths suggest possible chromatic effects or wavelength-dependent biases in the detection system. To fully understand these effects, further investigation with additional wavelengths and more precise instrumentation would be necessary.

Optical Alignment: Despite efforts to place the cuvette flush with the diffraction grating, any residual air gap introduces additional optical path considerations not fully captured by equation (9). The equation assumes perfect contact and parallel beam propagation.

Material Properties: The refractive indices of complex liquids like EVOO may vary with temperature, purity, and wavelength. These factors were not controlled or measured in this experiment, potentially contributing to discrepancies between measured and theoretical values. In particular, absorption and scattering within EVOO could distort the diffraction pattern, leading to inaccurate fringe position measurements. However, this effect is unlikely to fully account for the large underestimation observed, as EVOO is relatively transparent at the wavelengths used.

4.5 Sensitivity and Potential Applications

Despite the quantitative challenges, the experiment successfully demonstrated the ability to distinguish between liquids with similar refractive indices. Water ($n \approx 1.333$) and ethanol ($n \approx 1.361$) were clearly differentiated, with water measurements consistently yielding values around 1.32–1.44 and ethanol around 1.22–1.75. This separation indicates a sensitivity to refractive index changes of at least $\Delta n \approx 0.03$, or roughly 5×10^{-3}.

The experimental approach employed here is conceptually similar to the diffraction-based refractometry technique described by Durán-Ramírez et al. [1], which demonstrated that diffraction patterns can be effectively used to measure refractive indices of transparent media. While their implementation achieved higher precision using more sophisticated detection systems, our results support the same fundamental principle: that lateral displacement of diffraction fringes provides a reliable method for characterizing optical properties of liquids. The agreement between our findings and the established methodology validates the feasibility of this approach even with affordable, Arduino-based instrumentation.

Based on the observations and theoretical calculations, with optimization the LDR should be able to detect a change of at least 5×10^{-4} in the refractive index. For binary mixtures where the refractive index varies linearly with composition, this would correlate to an ability to detect approximately 2% impurity in a substance. While this wouldn't qualify as lab or medical grade purity, simple modifications could be made to increase the sensitivity to detect a change in refractive index down to 1×10^{-6}. Such improvements might include: (i) a higher-resolution position sensor to replace the stepper motor system, (ii) a photodiode array for simultaneous multipoint detection, (iii) temperature stabilization to eliminate thermal drift, and (iv) automated fringe-fitting algorithms to remove subjectivity from peak identification. With these enhancements, this process could potentially be used to determine very minute impurities in transparent or translucent liquids and likely gases, offering a low-cost alternative to commercial refractometers for applications in quality control, chemical analysis, and educational demonstrations.

5 Conclusion

Overall, the experiment demonstrates that a simple optical setup, supported by accessible Arduino-based instrumentation, can determine refractive indices across a range of liquids. The LDR-based detection system was able to distinguish between liquids with similar refractive indices, demonstrating sensitivity to changes on the order of $\Delta n \approx 0.03$. Using the optimized quartz macro cuvette setup, water was calculated to have a refractive index of 1.424 ± 0.211 (532 nm laser) with a percent error of 6.8% from the theoretical value of 1.333. Ethanol measurements ranged from 1.220 ± 0.077 to 1.533 ± 0.197 depending on wavelength and setup, yielding percent errors between 10.4% and 12.6% from the theoretical value of 1.361. Extra virgin olive oil was determined to be 1.175 ± 0.140 (532 nm) with approximately 20% underestimation from the theoretically accepted range of 1.465–1.480.

While the quantitative accuracy was limited by systematic errors in fringe detection and optical alignment, the experiment successfully validated the fundamental principle of diffraction-based refractometry and demonstrated the feasibility of low-cost instrumentation for optical characterization. The results highlight both the promise and the challenges of Arduino-based precision optics measurements, suggesting that with fur-

ther refinement of the detection algorithm and mechanical stability, this approach could provide a practical educational tool and potentially a viable alternative for industrial applications requiring moderate precision.

References

[1] Víctor M. Durán-Ramírez, Alejandro Martínez-Ríos, J. Ascención Guerrero-Viramontes, Jesús Muñoz-Maciel, Francisco G. Peña-Lecona, Romeo Selvas-Aguilar, and Gilberto Anzueto-Sánchez. Diffraction-based refractometry. *Optics Express*, 22(25):29899, 2014.

A Supplementary Code

This section contains the code used in the experiment for data acquisition and analysis.

A.1 Arduino Code for Stepper Motor and LDR Control

The Arduino code controls the stepper motor to move the LDR across the diffraction pattern and records intensity measurements.

A.1.1 Main Stepper Motor Control Code

```
#include <Stepper.h>
#include "CSVPrinter.h"

const int laserPin = 3;
const int ldrPin = A0;
const int numSamples = 20;

// Stepper constants
const int SPR = 32;
const int GEAR_RED = 64;
const int TOTAL_STEPS = 1800;  // SPR * GEAR_RED;
const int STEP_INCREMENT = 8;

Stepper stepperMotor(SPR, 8, 10, 9, 11);

struct LDRStats {
    float mean;
    float stddev;
};

LDRStats averageReadLDR(int pin, int numSamples) {
    float total = 0;
    float values[20]; // max 20 samples for memory safety
    if (numSamples > 20) numSamples = 20;
    for (int i = 0; i < numSamples; i++) {
        values[i] = analogRead(pin);
        total += values[i];
        delay(10); // Small delay to allow ADC to stabilize
    }
    float mean = total / (float)numSamples;
```

```
    float sumSq = 0;
    for (int i = 0; i < numSamples; i++) {
        sumSq += (values[i] - mean) * (values[i] - mean);
    }
    // Use Bessel's correction for sample stddev
    float stddev = (numSamples > 1) ? sqrt(sumSq / (float)(numSamples -
        1)) : 0;
    LDRStats stats = {mean, stddev};
    return stats;
}

void setup() {
    pinMode(laserPin, OUTPUT);
    digitalWrite(laserPin, HIGH); // Turn on laser
    stepperMotor.setSpeed(100); // Set speed to 100 RPM
    CSVPrinter::begin(9600);

    while (!Serial) {
        ; // Wait for Serial Monitor to open
    }

    CSVPrinter::printHeader("position,ldr_mean,ldr_stddev");
    delay(7000); // Allow time for laser to stabilize and for me to
       open the Serial Monitor
}

void loop() {
    static bool done = false;
    if (!done) {
        for (int step = 0; step <= TOTAL_STEPS; step += STEP_INCREMENT)
            {
            stepperMotor.step(STEP_INCREMENT); // Move stepper
            delay(50); // Allow time for motor to stabilize
            LDRStats stats = averageReadLDR(ldrPin, numSamples);
            CSVPrinter::printRow(step, stats.mean, stats.stddev);
            delay(50);
        }
        done = true;
        digitalWrite(laserPin, LOW); // Turn off laser
        stepperMotor.setSpeed(500); // Increase speed for return
        stepperMotor.step(-TOTAL_STEPS); // Return to start
    }
}
```

Listing 1: Spectrometer stepper motor control code

A.1.2 CSV Printer Library

```
#ifndef CSV_PRINTER_H
#define CSV_PRINTER_H

#include <Arduino.h>

/**
 * @brief A library for printing CSV formatted data to the Arduino
    Serial monitor.
 *
```

```
 * This library provides convenient functions for outputting data in CSV format,
 * making it easy to collect and analyze experimental data from Arduino projects.
 *
 * @author Generated for PHY3902 Labs
 * @version 1.0
 */

class CSVPrinter {
public:
    /**
     * @brief Initialize the CSV printer with Serial communication.
     * @param baudRate The baud rate for Serial communication (default: 9600)
     */
    static void begin(long baudRate = 9600);

    /**
     * @brief Prints a value in CSV format to the Serial output.
     * @tparam T The type of the value to print.
     * @param value The value to print.
     * @param eol If true, prints a newline after the value; otherwise, prints a comma.
     * @param precision Number of decimal places for floating point values (default: 4)
     */
    template <typename T>
    static void printValue(T value, bool eol = false, int precision = 4);

    /**
     * @brief Prints a CSV header row.
     * @param headers Array of header strings
     * @param numHeaders Number of headers in the array
     */
    static void printHeader(const char* headers[], int numHeaders);

    /**
     * @brief Prints a CSV header row using a single string with comma-separated values.
     * @param headerString String containing comma-separated header names
     */
    static void printHeader(const char* headerString);

    /**
     * @brief Starts a new CSV row (no output, just for organization)
     */
    static void startRow();

    /**
     * @brief Ends the current CSV row with a newline
     */
    static void endRow();

    /**
     * @brief Prints a complete CSV row with 2 values
```

```
     */
    template <typename T1, typename T2>
    static void printRow(T1 val1, T2 val2, int precision = 4);

    /**
     * @brief Prints a complete CSV row with 3 values
     */
    template <typename T1, typename T2, typename T3>
    static void printRow(T1 val1, T2 val2, T3 val3, int precision = 4);

    /**
     * @brief Prints a complete CSV row with 4 values
     */
    template <typename T1, typename T2, typename T3, typename T4>
    static void printRow(T1 val1, T2 val2, T3 val3, T4 val4, int
        precision = 4);
};

// Template function implementations

template <typename T>
void CSVPrinter::printValue(T value, bool eol, int precision) {
    if (eol) {
        Serial.println((float)value, precision);
    } else {
        Serial.print((float)value, precision);
        Serial.print(",");
    }
}

template <typename T1, typename T2>
void CSVPrinter::printRow(T1 val1, T2 val2, int precision) {
    printValue(val1, false, precision);
    printValue(val2, true, precision);
}

template <typename T1, typename T2, typename T3>
void CSVPrinter::printRow(T1 val1, T2 val2, T3 val3, int precision) {
    printValue(val1, false, precision);
    printValue(val2, false, precision);
    printValue(val3, true, precision);
}

template <typename T1, typename T2, typename T3, typename T4>
void CSVPrinter::printRow(T1 val1, T2 val2, T3 val3, T4 val4, int
    precision) {
    printValue(val1, false, precision);
    printValue(val2, false, precision);
    printValue(val3, false, precision);
    printValue(val4, true, precision);
}

// Non-template function implementations

void CSVPrinter::begin(long baudRate) {
    Serial.begin(baudRate);
}

```

```
void CSVPrinter::printHeader(const char* headers[], int numHeaders) {
    for (int i = 0; i < numHeaders; i++) {
        Serial.print(headers[i]);
        if (i < numHeaders - 1) {
            Serial.print(",");
        }
    }
    Serial.println();
}

void CSVPrinter::printHeader(const char* headerString) {
    Serial.println(headerString);
}

void CSVPrinter::startRow() {
    // This function is for organization/readability - no actual output
}

void CSVPrinter::endRow() {
    Serial.println();
}

#endif // CSV_PRINTER_H
```

Listing 2: CSV printer header file for data output

A.2 Python Code for Data Analysis

The Python code processes the intensity data and calculates the refractive indices.

A.2.1 Main Analysis Script

```
#!/usr/bin/env python3
"""
Intensity Plotting and Fringe Analysis Script

This script processes interferometry data from diffraction grating experiments,
analyzing fringe patterns to calculate refractive indices. It supports both
individual file processing and batch processing with automatic baseline
comparisons.

Usage
-----
Process single file:
    python intensity_plot.py filename.csv

Process all files in auto mode:
    python intensity_plot.py auto

Enable debug mode for detailed output:
    python intensity_plot.py auto debug
    python intensity_plot.py filename.csv debug

Skip displaying graphs (still saves them):
```

```
    python intensity_plot.py auto --no-show
    python intensity_plot.py filename.csv --no-show

Features
--------
- Automatic fringe detection and numbering
- Error propagation through all calculations
- Baseline comparison for refractive index determination
- Color-coded console output
- Debug mode with detailed fringe information
- Smart filtering (skips parent baselines in normal auto mode)
- Optional graph display control (--no-show flag)
"""
import json
from typing import Optional

from utils import FringeAnalysis, IntensityConfig
import pandas as pd
import matplotlib.pyplot as plt
import sys
import os
from colorama import Fore, Style, init

# Initialize colorama for cross-platform color support
init(autoreset=True)

CONSTANTS = IntensityConfig()


def plot_intensity(
    data_file: str, output_file: str, manifest: dict, debug: bool =
        False, show_plot: bool = True
):
    """
    Plot intensity data with fringe analysis and optional baseline
        comparison.

    Main processing function that loads data, performs fringe analysis,
    generates plots, and prints results. Handles both baseline parent
        files
    and child files with automatic baseline comparison.

    Parameters
    ----------
    data_file : str
        Name of the CSV data file to process
    output_file : str
        Name for the output plot file (will be placed in OUT_DIR)
    manifest : dict
        Manifest dictionary containing metadata for all datasets
    debug : bool, optional
        If True, print detailed debug information, by default False
    show_plot : bool, optional
        If True, display the plot window, by default True
    """
    output_file = os.path.join(CONSTANTS.OUT_DIR, output_file)
    data_path = os.path.join(CONSTANTS.DATA_DIR, data_file)

```

```
    if not os.path.exists(data_path):
        print(f"{Fore.YELLOW}Warning: Data file not found: {data_file}")
        return

    # Load and validate manifest entry
    manifest_entry = manifest.get(data_file, {})
    if not manifest_entry:
        print(
            f"{Fore.YELLOW}Warning: No manifest entry found for {data_file}."
        )
        return

    # Load data and perform basic analysis
    data = pd.read_csv(data_path)
    analyzer = FringeAnalysis(fringe_min_width=CONSTANTS.FRINGE_MIN_WIDTH)
    analyzer.analyze(data, manifest_entry, CONSTANTS)

    # Handle baseline analysis if applicable
    baseline_info = manifest_entry.get("baseline", None)
    if baseline_info:
        is_baseline = baseline_info.get("parent", False)

        if not is_baseline:
            # This is a child dataset - compare to baseline
            parent_baseline_name = baseline_info.get("name")
            parent_file = _find_baseline_file(manifest, parent_baseline_name)

            if parent_file:
                baseline_analyzer = _analyze_baseline(parent_file, manifest)
                if baseline_analyzer:
                    analyzer.analyze_baseline_child(
                        baseline_analyzer, manifest_entry, CONSTANTS
                    )

    # Generate plot
    _create_plot(analyzer, data_file, show_plot)

    # Print colorized summary
    _print_summary(analyzer, data_file, debug)


def _find_baseline_file(manifest: dict, baseline_name: str) -> Optional[str]:
    """Find the parent baseline file in the manifest.

    Parameters
    ----------
    manifest : dict
        Manifest dictionary containing all dataset metadata
    baseline_name : str
        Name identifier of the baseline to find

    Returns
```

```
    -------
    filename : str or None
        Filename of the parent baseline, or None if not found
    """
    for fname, entry in manifest.items():
        binfo = entry.get("baseline", {})
        if binfo.get("name") == baseline_name and binfo.get("parent",
            False):
            return fname
    return None


def _analyze_baseline(
    baseline_file: str, manifest: dict
) -> Optional[FringeAnalysis]:
    """Load and analyze baseline data.

    Parameters
    ----------
    baseline_file : str
        Filename of the baseline dataset
    manifest : dict
        Manifest dictionary containing all dataset metadata

    Returns
    -------
    baseline_analyzer : FringeAnalysis or None
        Analyzer instance with baseline fringe data, or None if file
            not found
    """
    baseline_data_path = os.path.join(CONSTANTS.DATA_DIR, baseline_file
        )

    if not os.path.exists(baseline_data_path):
        print(
            f"{Fore.YELLOW}Warning: Baseline file not found: {
                baseline_file}"
        )
        return None

    baseline_data = pd.read_csv(baseline_data_path)
    baseline_manifest_entry = manifest.get(baseline_file, {})
    baseline_analyzer = FringeAnalysis(
        fringe_min_width=CONSTANTS.FRINGE_MIN_WIDTH
    )
    baseline_analyzer.analyze(
        baseline_data, baseline_manifest_entry, CONSTANTS
    )

    return baseline_analyzer


def _create_plot(analyzer: FringeAnalysis, data_file: str, show_plot:
    bool = True):
    """Create and save the intensity plot.

    Generates a matplotlib figure showing intensity vs position with
    highlighted fringe regions and error bars.
```

```
183
184     Parameters
185     ----------
186     analyzer : FringeAnalysis
187         Analyzer instance containing processed fringe data
188     data_file : str
189         Name of the data file (used for output filename)
190     show_plot : bool, optional
191         If True, display the plot window, by default True
192     """
193     plt.figure(figsize=(10, 6))
194
195     # Plot intensity with error bars
196     plt.errorbar(
197         analyzer.position,
198         analyzer.intensity,
199         yerr=analyzer.intensity_error,
200         label="LDR Feedback",
201         color="magenta",
202         markersize=2,
203         fmt="o",
204         ecolor="thistle",
205         elinewidth=1,
206         capsize=2,
207     )
208
209     # Highlight fringe regions
210     fringe_regions = analyzer.center_fringe_regions + analyzer.
        filtered_outer
211     for idx, (start_idx, end_idx) in enumerate(fringe_regions):
212         plt.axvspan(
213             analyzer.position[start_idx],
214             analyzer.position[end_idx],
215             color="yellow",
216             alpha=0.3,
217             label="Fringe Region" if idx == 0 else None,
218         )
219
220     plt.title("LDR Feedback vs Position")
221     plt.xlabel("Position (m)")
222     plt.ylabel("LDR Proportional Feedback")
223     plt.grid(True)
224     plt.legend()
225
226     # Save plot
227     output_file = os.path.join(
228         CONSTANTS.OUT_DIR, os.path.splitext(data_file)[0] + "_intensity
            .png"
229     )
230     os.makedirs(CONSTANTS.OUT_DIR, exist_ok=True)
231     plt.savefig(output_file)
232     if show_plot:
233         plt.show()
234     plt.close()
235
236
237 def _print_summary(
238     analyzer: FringeAnalysis, data_file: str, debug: bool = False
```

```
):
    """Print a colorized, formatted summary of the analysis.

    Outputs a formatted console summary with color-coded sections highlighting
    key results including detected fringes and refractive index calculations.

    Parameters
    ----------
    analyzer : FringeAnalysis
        Analyzer instance containing processed fringe data
    data_file : str
        Name of the data file being analyzed
    debug : bool, optional
        If True, include debug information in output, by default False
    """
    # Header
    print(f"\n{Style.BRIGHT}{Fore.CYAN}{'='*70}")
    print(f"{Style.BRIGHT}{Fore.CYAN}  Analysis: {Fore.MAGENTA}{data_file}")
    print(f"{Style.BRIGHT}{Fore.CYAN}{'='*70}")

    # Notes
    if analyzer.notes:
        print(f"{Style.DIM}  {analyzer.notes}\n")

    # Get the summary text and colorize it
    summary = analyzer.summarize_fringe_analysis(debug=debug)

    # Colorize different sections
    lines = summary.split("\n")
    for line in lines:
        if line.startswith("[DEBUG]"):
            # Debug info in dim yellow
            print(f"{Style.DIM}{Fore.YELLOW}{line}")
        elif line.startswith("Detected Fringes:"):
            # Section header in bold green
            print(f"{Style.BRIGHT}{Fore.GREEN}{line}")
        elif line.startswith("Refractive Index Results:"):
            # Section header in bold blue
            print(f"{Style.BRIGHT}{Fore.BLUE}{line}")
        elif "Order" in line and ":" in line:
            # Fringe/order lines - highlight the order number and value
            if "distance" in line:
                # Fringe distance line
                parts = line.split("distance = ")
                if len(parts) == 2:
                    order_part = parts[0]
                    value_and_rest = parts[1].split(" ", 1)
                    value = value_and_rest[0]
                    rest = value_and_rest[1] if len(value_and_rest) > 1 else ""
                    print(
                        f"{order_part}{Style.BRIGHT}{Fore.YELLOW}distance = {value}{Style.RESET_ALL} {Style.DIM}{rest}"
                    )
```

```
                else:
                    print(line)
            elif "n␣=" in line:
                # Refractive index line
                parts = line.split("n␣=␣")
                if len(parts) == 2:
                    print(f"{parts[0]}{Style.BRIGHT}{Fore.CYAN}n␣=␣{
                        parts[1]}")
                else:
                    print(line)
            else:
                print(line)
        elif line.strip().startswith("Mean:"):
            # Mean value in bold magenta
            parts = line.split("n␣=␣")
            if len(parts) == 2:
                print(f"{parts[0]}{Style.BRIGHT}{Fore.MAGENTA}n␣=␣{
                    parts[1]}")
            else:
                print(f"{Style.BRIGHT}{Fore.MAGENTA}{line}")
        else:
            print(line)

    print(f"{Style.BRIGHT}{Fore.CYAN}{'='*70}\n")


if __name__ == "__main__":
    # Load manifest
    with open(CONSTANTS.MANIFEST_PATH, "r", encoding="utf-8") as f:
        manifest = json.load(f)

    # Parse command line arguments
    auto_mode = False
    debug_mode = False
    show_plots = True
    target_file = None

    # Check for debug and no-show flags
    args = [arg.lower() for arg in sys.argv[1:]]
    if "debug" in args:
        debug_mode = True
        args = [arg for arg in args if arg != "debug"]

    if "--no-show" in args:
        show_plots = False
        args = [arg for arg in args if arg != "--no-show"]

    if len(args) > 0:
        if "auto" in args:
            auto_mode = True
        else:
            target_file = (
                sys.argv[1]
                if sys.argv[1].lower() not in ["debug", "--no-show"]
                else (sys.argv[2] if len(sys.argv) > 2 and sys.argv[2].
                    lower() not in ["debug", "--no-show"] else (sys.argv
                    [3] if len(sys.argv) > 3 else None))
            )
```

```
345     else:
346         # Interactive mode
347         print(
348             f"{Style.BRIGHT}{Fore.CYAN}=== Intensity Plot Analysis ==={
                Style.RESET_ALL}"
349         )
350         print("Options: 'auto', 'debug', filename, or 'exit' to quit\n"
            )
351 
352         user_input = (
353             input("Enter your choice (or 'exit'/'cancel' to quit): ")
354             .strip()
355             .lower()
356         )
357 
358         # Check for exit commands
359         if user_input in ["exit", "quit", "cancel", "q"]:
360             sys.exit(0)
361 
362         if "debug" in user_input:
363             debug_mode = True
364             user_input = user_input.replace("debug", "").strip()
365 
366         if user_input == "auto":
367             auto_mode = True
368         else:
369             target_file = user_input if user_input else None
370 
371             # If no file provided, prompt again with exit option
372             if not target_file:
373                 target_file = input("File name (or 'exit' to quit): ").
                    strip()
374 
375                 # Check for exit on second prompt
376                 if target_file.lower() in ["exit", "quit", "cancel", "q
                    "]:
377                     sys.exit(0)
378 
379     # Execute based on mode
380     if auto_mode:
381         print(
382             f"{Style.BRIGHT}{Fore.GREEN}Processing all files from
                manifest..."
383         )
384         if debug_mode:
385             print(f"{Fore.YELLOW}Debug mode: ON (processing all files)\
                n")
386         else:
387             print(
388                 f"{Style.DIM}(Skipping parent files and files without
                    baseline)\n"
389             )
390 
391         for fname in manifest.keys():
392             manifest_entry = manifest.get(fname, {})
393             baseline_info = manifest_entry.get("baseline", None)
394 
395             # Skip logic (only in non-debug mode)
```

```
            if not debug_mode:
                if baseline_info:
                    is_parent = baseline_info.get("parent", False)
                    if is_parent:
                        print(
                            f"{Style.DIM}Skipped {fname}: Parent baseline file"
                        )
                        continue
                else:
                    print(f"{Style.DIM}Skipped {fname}: No baseline reference")
                    continue

            output_file = os.path.splitext(fname)[0] + "_intensity.png"
            plot_intensity(fname, output_file, manifest, debug=debug_mode, show_plot=show_plots)
    else:
        if target_file:
            output_file = os.path.splitext(target_file)[0] + "_intensity.png"
            if debug_mode:
                print(f"{Fore.YELLOW}Debug mode: ON")
            plot_intensity(
                target_file, output_file, manifest, debug=debug_mode, show_plot=show_plots
            )
        else:
            print(f"{Fore.RED}Error: No file specified")
```

Listing 3: Intensity plotting and analysis code

A.2.2 Theoretical Model Script

```
#!/usr/bin/env python3
"""
Theoretical Diffraction Pattern Plotter

Generates theoretical diffraction patterns for comparison with experimental
measurements from a diffraction grating interferometry setup.

Usage
-----
    python theoretical_diffraction.py
"""

import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

# Physical Constants
GRATING_LINES_PER_METER = 527559
GRATING_SPACING = 1.0 / GRATING_LINES_PER_METER  # meters

# Default Experimental Parameters
DEFAULT_WAVELENGTH = 650e-9  # meters (650 nm red laser)
```

```python
DEFAULT_SCREEN_DISTANCE = 0.045  # meters (4.5 cm)
DEFAULT_TRACK_RANGE = (-0.030, 0.030)  # meters (-30 to +30 mm)
DEFAULT_PEAK_WIDTH = 0.008  # meters
DEFAULT_BACKGROUND = 0.02  # normalized background intensity
DEFAULT_ORDER_RANGE = (-2, 3)  # diffraction orders to calculate

# Plot Parameters
DEFAULT_DPI = 300
OUTPUT_DIR = Path(__file__).parent.parent / 'out' / 'theoretical'


def calculate_diffraction_positions(wavelength, screen_distance,
                                    grating_spacing=GRATING_SPACING,
                                    order_range=DEFAULT_ORDER_RANGE):
    """
    Calculate theoretical positions of diffraction orders on the screen.

    Uses the grating equation: d sin(  ) = m

    Parameters
    ----------
    wavelength : float
        Wavelength of light in meters
    screen_distance : float
        Distance from grating to screen/detector in meters
    grating_spacing : float, optional
        Distance between grating lines in meters
    order_range : tuple, optional
        Range of diffraction orders to calculate (start, stop)

    Returns
    -------
    dict : Dictionary mapping order number to position in meters
    """
    positions = {}

    for m in range(*order_range):
        sin_theta = (m * wavelength) / grating_spacing

        if abs(sin_theta) <= 1.0:
            theta = np.arcsin(sin_theta)
            position = screen_distance * np.tan(theta)
            positions[m] = position

    return positions


def create_diffraction_pattern(wavelength, screen_distance,
                               track_range=DEFAULT_TRACK_RANGE,
                               peak_width=DEFAULT_PEAK_WIDTH,
                               background=DEFAULT_BACKGROUND,
                               n_points=2000):
    """
    Create a realistic diffraction intensity pattern as measured by an LDR.

```

```
    Models broad, Gaussian-shaped peaks characteristic of LDR measurements.

    Parameters
    ----------
    wavelength : float
        Wavelength in meters
    screen_distance : float
        Distance from grating to detector in meters
    track_range : tuple, optional
        Range of the track in meters (start, end)
    peak_width : float, optional
        Characteristic width of each peak (FWHM) in meters
    background : float, optional
        Background intensity level (0-1)
    n_points : int, optional
        Number of points in position array

    Returns
    -------
    position : np.ndarray
        Position array in meters
    intensity : np.ndarray
        Normalized intensity values
    peak_positions : dict
        Dictionary mapping order to peak position in meters
    """
    position = np.linspace(*track_range, n_points)
    intensity = np.zeros_like(position)

    # Calculate theoretical peak positions
    peak_positions = calculate_diffraction_positions(wavelength, screen_distance)

    # Standard deviation for Gaussian (   = FWHM / 2.355)
    sigma = peak_width / 2.355

    # Add Gaussian peak for each order within track range
    for order, pos in peak_positions.items():
        if track_range[0] <= pos <= track_range[1]:
            # Zeroth order has maximum intensity; others decrease with order
            amplitude = 1.0 if order == 0 else 0.5 / abs(order)
            intensity += amplitude * np.exp(-((position - pos) / sigma) ** 2)

    # Normalize and add background
    intensity = intensity / np.max(intensity) * (1 - background) + background

    return position, intensity, peak_positions


def plot_theoretical_pattern(wavelength=DEFAULT_WAVELENGTH,
                             screen_distance=DEFAULT_SCREEN_DISTANCE,
                             track_range=DEFAULT_TRACK_RANGE,
                             show_plot=True):
    """
```

```
    Plot theoretical diffraction pattern matching experimental setup.

    Parameters
    ----------
    wavelength : float, optional
        Wavelength of laser in meters
    screen_distance : float, optional
        Distance from grating to detector in meters
    track_range : tuple, optional
        Range of track in meters (start, end)
    show_plot : bool, optional
        Whether to display the plot
    """
    # Generate the pattern
    position, intensity, peak_positions = create_diffraction_pattern(
        wavelength=wavelength,
        screen_distance=screen_distance,
        track_range=track_range
    )

    # Create plot
    fig, ax = plt.subplots(figsize=(10, 6))
    ax.plot(position, intensity, 'b-', linewidth=1.5, label='
       Theoretical Pattern')

    # Mark theoretical peak positions
    for order, pos in peak_positions.items():
        if track_range[0] <= pos <= track_range[1]:
            ax.axvline(pos, color='red', linestyle='--', alpha=0.3,
               linewidth=1)
            ax.text(pos, 1.02, f'm={order:+d}', ha='center', fontsize
               =9, color='red')

    ax.set_xlabel('LDR Position (m)', fontsize=12)
    ax.set_ylabel('Normalized Intensity', fontsize=12)
    ax.set_title(
        f'Theoretical Diffraction Pattern\n'
        f'Grating: {GRATING_LINES_PER_METER} lines/m, '
        f'  = {wavelength*1e9:.1f} nm, '
        f'L = {screen_distance*100:.1f} cm',
        fontsize=13
    )
    ax.grid(True, alpha=0.3)
    ax.set_ylim(0, 1.1)
    ax.set_xlim(track_range)
    ax.legend(fontsize=10)

    plt.tight_layout()

    # Save plot
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    output_file = OUTPUT_DIR / 'theoretical_diffraction.png'
    plt.savefig(output_file, dpi=DEFAULT_DPI, bbox_inches='tight')

    # Print peak positions
    print('\n' + '=' * 60)
    print('Theoretical Peak Positions:')
    print('=' * 60)
```

```
    print(f'Screen distance: {screen_distance*100:.1f} cm')
    print(f'Wavelength: {wavelength*1e9:.1f} nm')
    print('-' * 60)
    for order in sorted(peak_positions.keys()):
        pos = peak_positions[order]
        if track_range[0] <= pos <= track_range[1]:
            print(f'  Order m = {order:+2d}: {pos:8.4f} m  ({pos
                *1000:7.2f} mm)')
    print('=' * 60)
    print(f'\nPlot saved to: {output_file}')

    if show_plot:
        plt.show()

    plt.close()

if __name__ == '__main__':
    plot_theoretical_pattern()
```

Listing 4: Theoretical intensity plotting

A.2.3 Utility Modules

```
#!/usr/bin/env python3
"""
Physics Experiment Utilities Package

This package provides shared utilities and base classes for physics
   experiment
analysis, specifically designed for interferometry experiments using
diffraction gratings.

Modules
-------
data_processing
    Data conversion and processing utilities including ADC to intensity
    conversion with automatic error propagation, position calculations,
    and refractive index calculations from baseline measurements.

fringe_analysis
    Fringe detection, numbering, and analysis utilities for
       interference
    patterns. Includes methods for baseline comparison and refractive
    index determination.

Classes
-------
DataProcessing
    Main class for processing experimental data from ADC readings to
    physical measurements with automatic error propagation.

FringeAnalysis
    Main class for analyzing interference fringe patterns, detecting
    fringes, assigning order numbers, and calculating refractive
       indices.

IntensityConfig
```

```
    Configuration settings for intensity data processing and plotting,
    including data directories, experimental constants, and thresholds.

Examples
--------
Basic usage for processing intensity data:

>>> from utils import DataProcessing, FringeAnalysis, IntensityConfig
>>> constants = IntensityConfig()
>>> data_proc = DataProcessing(r_series=9970, v_in=5.0, resolution
    =1023)
>>> analyzer = FringeAnalysis(fringe_min_width=10)
>>> # Load your data and analyze
>>> analyzer.analyze(data, manifest_entry, data_proc, constants)

Notes
-----
This package requires the following dependencies:
- numpy: numerical computations
- scipy: signal processing (connected component labeling)
- uncertainties: automatic error propagation
- pandas: data loading and manipulation
"""

from .data_processing import DataProcessing
from .fringe_analysis import FringeAnalysis
from .intensity_config import IntensityConfig

__all__ = [
    "DataProcessing",
    "FringeAnalysis",
    "IntensityConfig",
]
```

Listing 5: Package initialization

```
#!/usr/bin/env python3
"""
Data Processing Module

This module provides utilities for processing experimental data from
    the
interferometry experiment, including ADC conversion, error propagation,
and refractive index calculations.
"""
from typing import Tuple

import numpy as np
import uncertainties.unumpy as unp


class DataProcessing:
    """Process experimental data from ADC readings to physical
       measurements.

    This class handles the conversion of ADC values to intensity
       measurements
    with automatic error propagation, as well as geometric calculations
```

```
        for
    fringe analysis.

    Parameters
    ----------
    r_series : float, optional
        Series resistance in the voltage divider circuit (ohms), by
           default 9970
    v_in : float, optional
        Input voltage to the ADC (volts), by default 5.0
    resolution : int, optional
        ADC resolution (maximum count value), by default 1023

    Attributes
    ----------
    r_series : float
        Series resistance value
    v_in : float
        Input voltage value
    resolution : int
        ADC resolution value
    """

    def __init__(
        self, r_series: float = 9970, v_in: float = 5.0, resolution:
           int = 1023
    ):
        self.r_series = r_series
        self.v_in = v_in
        self.resolution = resolution

    def intensity_with_error(
        self, adc_values, adc_stddev
    ) -> Tuple[np.ndarray, np.ndarray]:
        """Convert ADC values to intensity with automatic error
           propagation.

        Uses the uncertainties package to automatically propagate
           errors through
        the complete transformation chain: ADC     Voltage
           Resistance     Intensity.
        The transformation follows the voltage divider formula for an
           LDR circuit:

        V_LDR = ADC * V_in / resolution
        R_LDR = (R_series * V_LDR) / (V_in - V_LDR)
        Intensity = 1 / R_LDR

        Parameters
        ----------
        adc_values : array_like
            Array of ADC readings (counts)
        adc_stddev : array_like
            Standard deviation of ADC readings (uncertainty in counts)

        Returns
        -------
        intensities : ndarray
```

```
            Calculated intensity values (1/ohms)
        intensity_errors : ndarray
            Propagated uncertainties in intensity values
        """
        # Clip ADC values to prevent division by zero at saturation
        # Maximum safe ADC value leaves a small margin below v_in
        max_safe_adc = (
            self.resolution * 0.999
        )  # 99.9% of max to avoid v_ldr     v_in
        adc_values_clipped = np.clip(adc_values, 0, max_safe_adc)

        # Create uncertainty-aware array from ADC values and their
           uncertainties
        adc_with_err = unp.uarray(adc_values_clipped, adc_stddev)

        # Convert to voltage (error automatically propagated)
        voltages = adc_with_err * self.v_in / self.resolution

        # Calculate LDR resistance (error automatically propagated)
        # R_LDR = (R_series * V_LDR) / (V_in - V_LDR)
        r_ldr = (self.r_series * voltages) / (self.v_in - voltages)

        # Calculate intensity as 1/R_LDR (error automatically
           propagated)
        intensity_with_err = 1.0 / r_ldr

        # Extract nominal values and standard deviations
        intensities = unp.nominal_values(intensity_with_err)
        intensity_errors = unp.std_devs(intensity_with_err)

        return intensities, intensity_errors

    @staticmethod
    def steps_to_linear_position(
        step_counts, max_steps: int, linear_scale: float
    ) -> np.ndarray:
        """Convert stepper motor steps to linear position.

        Parameters
        ----------
        step_counts : array_like
            Array of step counts from the stepper motor
        max_steps : int
            Maximum number of steps for full range
        linear_scale : float
            Linear distance corresponding to max_steps (meters)

        Returns
        -------
        positions : ndarray
            Linear positions in meters

        Examples
        --------
        >>> steps = np.array([0, 900, 1800])
        >>> positions = DataProcessing.steps_to_linear_position(steps,
           1800, 0.05)
        >>> positions
```

```
        array([0.   , 0.025, 0.05 ])
        """
        return (step_counts / max_steps) * linear_scale

    @staticmethod
    def get_region_center_idx(region: Tuple[int, int]) -> int:
        """Get the center index of a fringe region.

        Helper method to calculate the midpoint index of a region tuple,
        ensuring the result is a Python integer for array indexing.

        Parameters
        ----------
        region : tuple of (int, int)
            Region boundaries as (start_index, end_index)

        Returns
        -------
        center_idx : int
            Index at the center of the region
        """
        return int((int(region[0]) + int(region[1])) // 2)

    def align_center_fringe(
        self, position: np.ndarray, center_fringe_region: Tuple[int, int]
    ) -> np.ndarray:
        """Shift position array so the center fringe is at position zero.

        Parameters
        ----------
        position : ndarray
            Array of position values (meters)
        center_fringe_region : tuple of (int, int)
            Indices defining the center fringe region (start, end)

        Returns
        -------
        aligned_position : ndarray
            Position array shifted so center fringe center is at 0
        """
        center_idx = self.get_region_center_idx(center_fringe_region)
        center_pos = position[center_idx]
        return position - center_pos

    def get_fringe_distance(
        self,
        position: np.ndarray,
        center_fringe_region: Tuple[int, int],
        outer_fringe_regions: list[Tuple[int, int]],
    ) -> list[float]:
        """Calculate distances between center fringe and outer fringes.

        Parameters
        ----------
        position : ndarray
```

```
            Array of position values (meters)
        center_fringe_region : tuple of (int, int)
            Indices defining the center fringe region (start, end)
        outer_fringe_regions : list of tuple
            List of (start, end) index tuples for outer fringes

        Returns
        -------
        distances : list of float
            Distances from center fringe to each outer fringe (meters)
        """
        center_idx = self.get_region_center_idx(center_fringe_region)
        center_pos = position[center_idx]
        distances = []
        for region in outer_fringe_regions:
            outer_idx = self.get_region_center_idx(region)
            outer_pos = position[outer_idx]
            distances.append(abs(outer_pos - center_pos))
        return distances

    @staticmethod
    def calculate_refractive_index_from_baseline(
        fringe_order: int,
        fringe_diff: float,
        grating_period: float,
        cell_width: float,
        vacuum_wavelength: float,
    ) -> float:
        """Calculate refractive index from baseline fringe shift
           measurements.

        Uses the theoretical relationship between fringe shift and
           refractive
        index in a diffraction grating interferometer with a sample
           cell.

        Parameters
        ----------
        fringe_order : int
            Order number of the fringe (m)
        fringe_diff : float
            Difference in fringe position between baseline and sample (
               meters)
        grating_period : float
            Period of the diffraction grating (meters, = 1/
               lines_per_meter)
        cell_width : float
            Width of the sample cell (meters)
        vacuum_wavelength : float
            Wavelength of the laser in vacuum (meters)

        Returns
        -------
        n : float
            Calculated refractive index of the sample
        """
        m = fringe_order
        delta_d_x = fringe_diff
```

```
        T = grating_period
        w = cell_width
        lambda_0 = vacuum_wavelength

        term = 1 - (m**2 * lambda_0**2) / T**2
        numerator = w**2 * term
        denominator = (w * m * lambda_0 / T - delta_d_x * np.sqrt(term)
            ) ** 2

        n = (m * lambda_0 / T) * np.sqrt(1 + numerator / denominator)
        return n
```

Listing 6: Data processing utilities

```
#!/usr/bin/env python3
"""
Fringe Analysis Module

This module provides utilities for analyzing interference fringe
    patterns
from diffraction grating interferometry/spectrometry experiments. It
    handles fringe
detection, numbering, and refractive index calculations.
"""
from typing import Tuple, List

import numpy as np
import pandas as pd
from numpy import ndarray
from scipy.ndimage import label as sc_label
from .data_processing import DataProcessing
from .intensity_config import IntensityConfig


class FringeAnalysis:
    """Analyze interference fringe patterns from intensity data.

    This class provides methods to detect fringes in intensity profiles
        ,
    assign order numbers, calculate fringe separations, and compute
    refractive indices from baseline comparisons.

    Parameters
    ----------
    fringe_min_width : int, optional
        Minimum width (in data points) for a valid fringe region, by
            default 10

    Attributes
    ----------
    fringe_min_width : int
        Minimum width threshold for fringe detection
    position : ndarray or None
        Position array (meters), aligned so center fringe is at 0
    intensity : ndarray or None
        Measured intensity values
    intensity_error : ndarray or None
        Uncertainties in intensity measurements
```

```
    data_proc : DataProcessing
        Instance of DataProcessing for conversions
    center_fringe_regions : list or None
        List of (start, end) index tuples for center fringes
    outer_fringe_regions : list or None
        List of (start, end) index tuples for all detected fringes
    filtered_outer : list or None
        Outer fringes excluding those overlapping with center
    fringe_numbered : list or None
        List of (order, region) tuples with assigned fringe numbers
    fringe_distances : list or None
        Distances from center to each numbered fringe (meters)
    notes : str or None
        Experimental notes from manifest
    refractive_indices : list or None
        List of (order, n_value) tuples for baseline child calculations
    """

    def __init__(self, fringe_min_width: int = 10):
        self.fringe_min_width = fringe_min_width
        self.position = None
        self.intensity = None
        self.intensity_error = None
        self.data_proc = DataProcessing()
        self.center_fringe_regions = None
        self.outer_fringe_regions = None
        self.filtered_outer = None
        self.fringe_numbered = None
        self.fringe_distances = None
        self.notes = None
        self.refractive_indices = None  # Only for baseline child
            calculations

    def analyze(
        self,
        data: pd.DataFrame,
        manifest_entry: dict,
        constants: IntensityConfig,
    ):
        """Analyze fringe data and extract fringe regions and distances
            .

        Performs complete fringe analysis including position conversion
            ,
        intensity calculation with error propagation, fringe detection,
        filtering, alignment, numbering, and distance measurements.

        Parameters
        ----------
        data : pd.DataFrame
            Experimental data containing columns:
            - 'position': stepper motor step counts
            - 'ldr_mean': mean ADC values from LDR
            - 'ldr_stddev': standard deviation of ADC values
        manifest_entry : dict
            Manifest entry for this dataset containing:
            - 'notes': experimental notes (optional)
        constants : IntensityConfig
```

```
            Configuration object with attributes:
            - CENTER_FRINGE_THRESHOLD: threshold for center fringe detection
            - OUTER_FRINGE_THRESHOLD: threshold for outer fringe detection
        """
        self.notes = manifest_entry.get("notes", "")

        # Convert position steps to linear position
        self.position = self.data_proc.steps_to_linear_position(
            data["position"], max_steps=1800, linear_scale=0.05
        )

        # Calculate intensity with error propagation
        self.intensity, self.intensity_error = (
            self.data_proc.intensity_with_error(
                data["ldr_mean"], data["ldr_stddev"]
            )
        )

        # Find center and outer fringe regions
        self.center_fringe_regions = self.find_fringe_regions(
            self.intensity, threshold=constants.CENTER_FRINGE_THRESHOLD
        )
        self.outer_fringe_regions = self.find_fringe_regions(
            self.intensity, threshold=constants.OUTER_FRINGE_THRESHOLD
        )

        # Filter outer fringes to exclude those overlapping with center
        def is_outside_center(center, outer):
            for c_start, c_end in center:
                if not (outer[1] < c_start or outer[0] > c_end):
                    return False
            return True

        self.filtered_outer = [
            r
            for r in self.outer_fringe_regions
            if is_outside_center(self.center_fringe_regions, r)
        ]

        # Align position so center fringe is at position 0
        if self.center_fringe_regions:
            self.position = self.data_proc.align_center_fringe(
                self.position, self.center_fringe_regions[0]
            )

            # Assign fringe numbers
            self.fringe_numbered = self.assign_fringe_numbers(
                self.position,
                self.center_fringe_regions[0],
                self.filtered_outer,
            )

            # Calculate fringe distances
            self.fringe_distances = self.data_proc.get_fringe_distance(
                self.position,
                self.center_fringe_regions[0],
```

```
                [region for _, region in self.fringe_numbered],
            )
        else:
            self.fringe_numbered = []
            self.fringe_distances = []

    def analyze_baseline_child(
        self,
        baseline_analyzer,
        manifest_entry: dict,
        constants: IntensityConfig,
    ):
        """Calculate refractive indices for child dataset using
           baseline comparison.

        Compares fringe positions between this dataset (child) and a
           baseline
        dataset to calculate the refractive index of the sample. This
           method
        should only be called on child datasets that have a
           corresponding parent
        baseline.

        Parameters
        ----------
        baseline_analyzer : FringeAnalysis
            Analyzer instance containing baseline (reference) fringe
               data
        manifest_entry : dict
            Manifest entry containing:
            - 'wavelength': laser wavelength in vacuum (meters),
               optional
        constants : object
            Configuration object with attributes:
            - DIFFRACTION_GRATING: grating line density (lines/meter)
            - CUVETTE_WIDTH: sample cell width (meters)
        """
        grating_period = 1 / constants.DIFFRACTION_GRATING
        cell_width = constants.CUVETTE_WIDTH
        vacuum_wavelength = manifest_entry.get("wavelength", 650e-9)

        # Build dictionaries mapping fringe number to distance
        baseline_fringe_dict = {
            num: dist
            for (num, _), dist in zip(
                baseline_analyzer.fringe_numbered,
                baseline_analyzer.fringe_distances,
            )
        }
        child_fringe_dict = {
            num: dist
            for (num, _), dist in zip(
                self.fringe_numbered, self.fringe_distances
            )
        }

        # Find common fringe orders between baseline and child
        common_fringe_numbers = set(baseline_fringe_dict.keys()) & set(
```

```
            child_fringe_dict.keys()
        )

        self.refractive_indices = []
        for fringe_num in sorted(common_fringe_numbers):
            baseline_dist = baseline_fringe_dict[fringe_num]
            child_dist = child_fringe_dict[fringe_num]
            fringe_diff = baseline_dist - child_dist

            # Calculate refractive index from baseline difference
            n = self.data_proc.calculate_refractive_index_from_baseline
                (
                fringe_order=abs(fringe_num),
                fringe_diff=fringe_diff,
                grating_period=grating_period,
                cell_width=cell_width,
                vacuum_wavelength=vacuum_wavelength,
            )
            self.refractive_indices.append((fringe_num, n))

    def find_fringe_regions(self, intensity, threshold: float = 0.1):
        """Find regions of intensity peaks that represent fringes.

        Uses a threshold-based approach with connected component
           labeling
        to identify contiguous regions where intensity exceeds the
           background
        by a specified threshold.

        Parameters
        ----------
        intensity : array_like
            Array of intensity values
        threshold : float, optional
            Relative threshold above background for fringe detection,
               by default 0.1
            Threshold is applied as: background + threshold * (max -
               background)

        Returns
        -------
        regions : list of tuple
            List of (start_idx, end_idx) tuples defining fringe regions
            Only regions wider than `fringe_min_width` are included
        """
        background = np.median(intensity)
        fringe_mask = intensity > (
            background + threshold * (intensity.max() - background)
        )
        labeled, num_features = sc_label(fringe_mask)
        regions = []
        for i in range(1, num_features + 1):
            indices = np.where(labeled == i)[0]
            if len(indices) > self.fringe_min_width:
                regions.append((int(indices[0]), int(indices[-1])))
        return regions

    def assign_fringe_numbers(
```

```
        self,
        position: ndarray,
        center_fringe_region: Tuple[int, int],
        outer_fringe_regions: List[Tuple[int, int]],
    ) -> List[Tuple[int, tuple]]:
        """Assign order numbers to fringes based on their position
           relative to center.

        Fringes to the left of center are assigned negative orders (-1,
            -2, ...),
        fringes to the right are assigned positive orders (+1, +2, ...)
           , and the
        center fringe is assigned order 0 (implicitly, not included in
           output).

        Parameters
        ----------
        position : ndarray
            Array of position values (meters), should be aligned with
               center at 0
        center_fringe_region : tuple of (int, int)
            Indices defining the center fringe region (start, end)
        outer_fringe_regions : list of tuple
            List of (start, end) index tuples for outer fringes

        Returns
        -------
        numbered : list of tuple
            List of (order, region) tuples where:
            - order is an integer (negative for left, positive for
               right)
            - region is a (start, end) index tuple
            Sorted by order: [-2, -1, +1, +2, ...]
        """
        center_idx = self.data_proc.get_region_center_idx(
           center_fringe_region)
        center_pos = position[center_idx]
        numbered = []

        # Calculate center positions for outer fringes
        outer_centers = [
            (self.data_proc.get_region_center_idx(region), region)
            for region in outer_fringe_regions
        ]
        outer_centers.sort(key=lambda x: position[x[0]] - center_pos)

        # Separate left and right fringes
        left = [r for i, r in outer_centers if position[i] < center_pos
           ]
        right = [r for i, r in outer_centers if position[i] >
           center_pos]

        # Assign negative numbers to left fringes, positive to right
        for n, region in enumerate(reversed(left), 1):
            numbered.append((-n, region))
        for n, region in enumerate(right, 1):
            numbered.append((n, region))

```

```
306         return numbered
307
308     def summarize_fringe_analysis(self, debug: bool = False) -> str:
309         """Generate a summary of the fringe analysis results.
310
311         Creates a formatted text summary of the analysis including
               detected
312         fringes, their positions, distances, and calculated refractive
               indices
313         (if applicable).
314
315         Parameters
316         ----------
317         debug : bool, optional
318             If True, include detailed center and outer fringe
                   information
319             in the output, by default False
320
321         Returns
322         -------
323         summary : str
324             Multi-line formatted string containing:
325             - [Debug only] Center fringe position and range
326             - [Debug only] All outer fringe positions and ranges
327             - Detected fringes with order numbers and distances
328             - Refractive index results (if calculated from baseline)
329             - Mean and standard deviation of refractive indices
330
331         Examples
332         --------
333         Normal mode output:
334             Detected Fringes:
335               Order -1: distance = 19.3333 mm  [range: ...]
336               Order +1: distance = 19.7778 mm  [range: ...]
337
338             Refractive Index Results:
339               Order -1: n = 1.393251
340               Order +1: n = 2.105776
341
342               Mean: n = 1.749514    0.503831
343         """
344
345         summary = []
346
347         # Debug mode: Show detailed center and outer fringe information
348         if debug:
349             if self.center_fringe_regions:
350                 c_start, c_end = self.center_fringe_regions[0]
351                 c_center = self.data_proc.get_region_center_idx(
352                     self.center_fringe_regions[0]
353                 )
354                 summary.append(
355                     f"[DEBUG] Center fringe: {self.position[c_start]:.6
                           g} m to {self.position[c_end]:.6g} m "
356                     f"(center: {self.position[c_center]:.6g} m)"
357                 )
358
359             if self.filtered_outer:
```

```
                summary.append("\n[DEBUG] Outer fringes:")
                for region in self.filtered_outer:
                    start, end = region
                    center = self.data_proc.get_region_center_idx(
                        region)
                    summary.append(
                        f"  {self.position[start]:.6g} m to {self.
                            position[end]:.6g} m "
                        f"(center: {self.position[center]:.6g} m)"
                    )
                summary.append("")  # Add blank line after debug info

        # Numbered fringes with distances (focus on number and distance
            )
        if self.fringe_numbered:
            summary.append("Detected Fringes:")
            for (number, region), dist in zip(
                self.fringe_numbered, self.fringe_distances
            ):
                start, end = region
                center = self.data_proc.get_region_center_idx(region)
                summary.append(
                    f"  Order {number:+d}: distance = {dist*1000:.4f} 
                        mm  "
                    f"[range: {self.position[start]*1000:.2f} to {self.
                        position[end]*1000:.2f} mm, "
                    f"center: {self.position[center]*1000:.2f} mm]"
                )

        # Refractive indices (for baseline children)
        if self.refractive_indices:
            summary.append("\nRefractive Index Results:")
            for fringe_num, n in self.refractive_indices:
                summary.append(f"  Order {fringe_num:+d}: n = {n:.6f}")

            # Calculate mean and standard deviation
            n_values = [n for _, n in self.refractive_indices]
            mean_n = np.mean(n_values)
            std_n = np.std(n_values, ddof=1) if len(n_values) > 1 else
                0
            summary.append(f"\n  Mean: n = {mean_n:.6f}    {std_n:.6f}"
                )

        return "\n".join(summary)
```

Listing 7: Fringe analysis utilities

```
#!/usr/bin/env python3
"""
Configuration settings for intensity data processing and plotting.
"""

import os
from dataclasses import dataclass


@dataclass
class IntensityConfig:
```

```
    DATA_DIR: str = os.path.abspath(
        os.path.join("..", "data", "intensity_data")
    )
    OUT_DIR = os.path.abspath(os.path.join("..", "out", "
       intensity_plots"))
    MANIFEST_PATH = os.path.join(DATA_DIR, "manifest.json")
    R_SERIES: float = 9970  # Resistance in ohms
    V_IN: float = 5.0  # Input voltage
    RESOLUTION: int = 1023  # ADC resolution
    DIFFRACTION_GRATING: float = 527559  # lines per meter
    # Minimum width of fringe region in data points
    FRINGE_MIN_WIDTH: int = 10
    CENTER_FRINGE_THRESHOLD: float = (
        0.1  # Threshold for center fringe detection
    )
    OUTER_FRINGE_THRESHOLD: float = (
        0.0025  # Threshold for outer fringe detection
    )
    CUVETTE_WIDTH: float = 0.01  # Cuvette width in meters
```

Listing 8: Configuration settings